

SUYOG KARKI

Data Science Intern | Machine Learning & Deep Learning

✉ suyogkarki2@gmail.com ☎ +977 9860103707 📍 Tokha, Kathmandu

🌐 [linkedin.com/in/suyog-karki-8544a033a](https://www.linkedin.com/in/suyog-karki-8544a033a) 🐙 github.com/suyogkarki1 🔗 suyogkarki1.com.np

PROFILE

Data Science student (BSIT), currently in my 4th year. I've worked through the full ML pipeline — cleaning data, EDA, feature engineering, model building, and tuning — and recently finished deep learning (CNN, RNN, LSTM, transfer learning). Built a few projects and deployed them using Streamlit and Docker. Looking for an AI/ML internship where I can keep learning and contribute to real projects.

EDUCATION

Higher Secondary Education (+2) - Science, 2020 – 2022
Kathmandu Model College (KMC), Bagbazar
GPA: 3.35 / 4.0

Bachelor of Science in Information Technology (BSIT) - Data Science, 2023 – 2027
Kings College, Kathmandu (Affiliated with Westcliff University, California)
Currently in 4th Year
GPA: 3.54 / 4.0

TECHNICAL SKILLS

Languages : Python · SQL

Data: Pandas · NumPy · Excel

Visualization: Matplotlib · Seaborn · Power BI

Deployment / Tools: Streamlit · Docker · Git & GitHub · PostgreSQL

ML / Modeling: Scikit-learn · PyTorch · Optuna

Deep Learning: ANN · CNN · RNN · LSTM · Transfer Learning

Currently Learning: Computer Vision · NLP / Transformers

CERTIFICATIONS

- Supervised Learning with Scikit-learn | DataCamp | 2026
- Associate Data Scientist Certification | DataCamp | 2026
- SQL Associate | DataCamp | 2026

PROJECTS

Diabetic Retinopathy Severity Grading (APTOS 2019)

Python · PyTorch · torchvision · OpenCV · Scikit-learn · Pandas · NumPy · Kaggle

GitHub: <https://github.com/suyogkarki1/aptos-2019-blindness-detection.git> 

- Built a model that grades diabetic eye disease severity (**0–4**) on the **APTOS 2019 dataset** (**3,662** retina images) with strong **class imbalance** (**49%** healthy vs **5%** Severe).
- Used **transfer learning** with **EfficientNet-B3** and framed the task as **regression** (one severity score with **MSE loss**) instead of 5-class classification, so near-miss grades are penalised less under **quadratic weighted kappa**.

- Built the full **PyTorch pipeline**: a custom **Dataset** class, **augmentation** (flips, rotation, contrast), DataLoaders, mixed-precision training with a **cosine learning-rate** schedule, best-checkpoint saving, and **test-time augmentation** at prediction.
- Found the model scored well locally but dropped on Kaggle's hidden test set, a sign of **distribution shift**, and fixed it by adding ~**17,000** external retina images, keeping all diseased cases and sampling **8,000** healthy ones to reduce the **class imbalance**.
- Reached **0.915** kappa and **80%** accuracy on a held-out test set, and improved the Kaggle private score from **0.848** to **0.894** (winner: **0.936**).

Cardio — Heart Disease Risk Predictor

Python · Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn · Streamlit

GitHub: <https://github.com/suyogkarki1/Clinical-Risk-Prediction-Diabetes-Heart-Disease.git> 

Live: https://clinical-risk-prediction-diabetes-heart-disease.streamlit.app/Heart_Disease 

- Built a heart disease risk predictor on the **UCI Cleveland dataset** (302 records), using a scikit-learn pipeline that imputes missing values, scales numeric features, and one-hot encodes categorical ones, so raw patient inputs are handled end-to-end.
- Engineered **6 extra features** from the clinical data (age and cholesterol groups, a high-risk-age flag, and heart-rate ratios) to give the models more signal to learn from.
- Compared six models with 5-fold cross-validation and tuned each with **GridSearchCV**, keeping all preprocessing inside the folds to prevent data leakage, and handled class balance with **class weights**.
- Chose a tuned **Random Forest** for the best test accuracy and smallest train-test gap, reaching **0.86** ROC-AUC, **79%** accuracy, **0.83** precision, and **0.76** recall on the test set.
- Deployed it as an interactive **Streamlit web app** with a live risk meter driven by the model's probability output.

Chini — Diabetes Risk Predictor for Women

Python · Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn · SciPy · Streamlit · Docker

GitHub: https://github.com/suyogkarki1/diabetes_pipeline.git 

Live: <https://chinisugar.streamlit.app/> 

- Built a diabetes risk predictor for adult women on the **Pima Indians dataset** (768 records), using a scikit-learn pipeline that fills in missing values and scales features, so it works even with incomplete patient data.
- Ran **Welch's t-tests** comparing diabetic vs non-diabetic patients, using the t-statistic and p-value ($\alpha = 0.05$) to judge significance and **Cohen's d** for effect size, to find which measurements best separate the two groups.
- Compared five models with 5-fold cross-validation and tuned each with GridSearchCV (including L1/L2/ElasticNet regularization for Logistic Regression), keeping all preprocessing inside the folds to prevent data leakage.
- Chose **Logistic Regression** and scored on PR-AUC instead of accuracy because the data is imbalanced, reaching **0.81** ROC-AUC, **73%** accuracy, and **0.72** recall on the test set.
- Deployed it as an interactive Streamlit web app, containerized with Docker for easy setup.

Customer Churn Prediction

Python · Pandas · Scikit-learn · Matplotlib · Seaborn

GitHub: <https://github.com/suyogkarki1/customer-churn-prediction> 

- Analyzed a Kaggle dataset of 10,000 European bank customers to identify behavioral patterns linked to churn, including credit score, account balance, and product usage habits.

- Engineered new features including **BalancePerProduct** and **AgeGroup**; conducted full EDA to uncover class imbalance and key churn indicators before applying preprocessing steps.
- Achieved **86.35% accuracy and 0.87 ROC-AUC** using Gradient Boosting Classifier, selected after benchmarking against simpler models including Logistic Regression and Decision Tree.
- Feature importance analysis revealed **Age and NumOfProducts** as the strongest churn predictors — suggesting retention teams should prioritize older single-product customers as the highest risk segment

Kathmandu Rain Prediction

Python · Pandas · NumPy · Scikit-learn · Matplotlib · Seaborn · Streamlit

GitHub: <https://github.com/suyogkarki1/Rain-Prediction-> 

Live: <https://barshamausam.streamlit.app/> 

- Sourced a 10-year Kathmandu weather dataset (2015–2025, 3,468 records) from Kaggle, and performed data cleaning, unit conversion, and feature engineering across 14 meteorological variables for supervised classification.
- Built and evaluated four classification models Logistic Regression, Decision Tree, K-Nearest Neighbors, and Random Forest using stratified 6-fold cross-validation; selected Random Forest as the final model, achieving **84% accuracy and 0.90 ROC-AUC** on held-out test data.
- Conducted feature importance analysis identifying **humidity, temperature, and cloud cover** as the strongest rainfall predictors, validating the model against real-world Kathmandu Valley monsoon patterns.
- Applied **bootstrap resampling** to estimate 2026 monthly rain probabilities with lower and upper uncertainty bounds, and developed a separate **SARIMA** seasonal time-series model to forecast monthly rainy-day percentages for comparison.
- Deployed the trained model as an interactive Streamlit web application (**SkyCast**) enabling real-time rain classification from user-defined weather inputs and visualizing seasonal forecast trends.

UAE ID Card Number Extraction Pipeline

Python · YOLOv11 (Ultralytics) · OpenCV · Tesseract OCR · PostgreSQL

GitHub: https://github.com/suyogkarki1/object_detection.git 

- Annotated a custom dataset of 424 UAE ID card images on Ultralytics HUB, labeling the ID-number field as a dedicated detection class, split into 405 train / 10 validation / 9 test images.
 - Trained and deployed a custom YOLOv11 model via Ultralytics' hosted endpoint, achieving **99.5% mAP50** and **95.0% recall** on ID-number localization.
- Built an OCR pipeline (OpenCV preprocessing + Tesseract) to extract and clean detected ID numbers via regex, then persisted records to PostgreSQL with conflict-safe deduplication.

QR Code Validator Using Finite Automata

Python · Graphviz · Deterministic Finite Automata

GitHub: <https://github.com/suyogkarki1/qr-code-authentication.git> 

- Built a system that validates QR code data using a Deterministic Finite Automaton (DFA), checking the bitstream structure step by step and accepting or rejecting based on whether it reaches a valid end state.
- Parsed and extracted structured information from valid codes, handling two formats bank payment QRs (account name, number, type) and WiFi QRs (SSID, password).
- Generated DFA state diagrams with Graphviz to visualize the validation flow.
- Applied theory-of-computation concepts (finite automata) to a practical data-validation problem

LANGUAGES

Nepali
Native

English
Fluent